

Faculty of Engineering and Technology Electrical and Computer Engineering Department

ENCS4320 – Applied Cryptography

SecureVault

A Multi-User Encrypted Document Exchange System

Summer Semester 2025-2026

Prepared by:

Yaqeen Salameh # 1230011

Aseel Hamed #1231891

Ruaa Othman #1230014

Instructor: Dr. Ahamd Shawahnah

Section: 1

Date: 23/9/2026

Table of Contents

Table of Contents **2**

Introduction: **1**

1. Threat Model **2**

Out of scope 2

2. Design **2**

2.1 Architecture 2

2.2 Key Hierarchy 3

2.3 Byte-Level Object and Message Formats 3

2.4 Life of a Document: Upload to Verified Retrieval 4

3. Justification of Section 7 Decisions **5**

3.1 Protecting Passwords 5

3.2 Protecting Document Contents 6

3.3 Public-Key Cryptography 6

3.4 Trusting a Public Key 6

3.5 Signatures Versus MACs 7

3.6 Architecture and Formats 7

Implementation / Library Declaration 7

4. Security Argument **7**

Correctness Evidence 8

5. Limitations **9**

6. AI Usage Statement and Contributions **10**

6.1 AI Usage Statement 10

6.2 Contribution Statements 10

References **10**

Introduction:

SecureVault is a secure document sharing system that allows multiple users to store and share documents with each other. The main idea of the project is to make sure that a document can be shared between users without exposing its content or important keys to unauthorized parties.

In SecureVault, users can protect their documents and share them with specific users. The system also needs to make sure that the information being exchanged has not been changed and that an old valid message cannot simply be reused. This is especially important when the server is not fully trusted.

The project focuses on building these security features as part of one complete system. It combines document protection, secure key sharing, user trust, authentication, and protection against modified or replayed data.

1. Threat Model

Our threat model follows the project specification. The client machine of an honest user is trusted. The server follows the protocol but is honest-but-curious: it can read everything sent to it, and its stored database may later be stolen. The network is considered attacker-controlled.

Party	Capabilities / assumptions
Users	Run the SecureVault client on a trusted machine. The attacker cannot read an honest user's memory or disk.
Server	Stores accounts and encrypted objects and relays messages. It follows the protocol, but it can read everything given to it. Its full database may be stolen.
Attacker	Can observe and modify network traffic, replay old messages, register accounts, and obtain a full copy of the server's stored data.

Out of scope

- Malware or compromise of an honest user's own machine.
- Coercion of a user.
- Traffic analysis based on message sizes and timing.
- Denial-of-service attacks.

This model is important because it decides where secrets are allowed to exist. The server does not receive the password, password-derived wrap key, private identity keys in plaintext, or plaintext document keys.

2. Design

2.1 Architecture

The implementation uses a client-server architecture with a shared directory that simulates the network. The client runs commands and keeps local state. The transport module writes message files into per-recipient inboxes. The server reads the server inbox, dispatches each object or account message, and stores account data and encrypted objects.

Component	Role

client.py	Registration, login, upload, download, sharing, receiving shares, password change, and local state.
transport.py	Shared-directory message delivery using temporary files followed by rename.
server.py	Dispatches account messages and stored-object requests.
server_private/	Server-side SQLite account database.
server_storage/	Encrypted Document objects and Share objects.
client_local/	Per-user session state, trusted-key state, freshness state, and protected local document keys.

2.2 Key Hierarchy

One password is not used directly as a cryptographic key. The client first runs Argon2id, then uses HKDF-SHA-256 with separate labels so authentication and private-key wrapping use different 32-byte values.

Source	Derived / generated value	Purpose
Password + 16-byte salt	Argon2id output (32 B)	High-cost password-derived input material.
Argon2id output	HKDF label securevault: v1:auth → auth value (32 B)	Authentication side; SHA256(auth value) is stored as the verifier.
Argon2id output	HKDF label securevault: v1:wrap → wrap key (32 B)	Encrypts the private identitykey bundle and protects local document keys.
Successful login	HKDF (login key, session ID, securevault: v1:session-key)	Per-session request authentication key.
Upload	Fresh document key (32 B)	ChaCha20-Poly1305 encryption of one document.
Sharing	P-256 ECDH secret → HKDF securevault: v1:share-wrap	Wraps the 32-byte document key for the named recipient.

2.3 Byte-Level Object and Message Formats

Stored cryptographic objects use version 1 and a one-byte type. Variable-length byte fields use a 4-byte big-endian length prefix. Integer fields such as size, timestamp, and sequence number are encoded by big-endian.

Object	Type	Fields after version/type header

Credential	1	user_id; salt (16); Argon2 params; stored verifier(32)
Public Key	2	user_id; P-256 public (65); RSA modulus(384); RSA exponent(3)
WrappedPrivateKey	3	user_id; nonce (12); ciphertext; tag(16)
Document	4	document_id(16); owner; filename; MIME type; size(8); timestamp(8); nonce (12); ciphertext; tag(16)
Share	5	document_id(16); sender; recipient; two fingerprints(32 each); sequence (8); timestamp(8); ephemeral P-256 public(65); wrapped key; RSA-PSS signature(384)

- Application account messages use version 1. The byte envelope consists of version (1 B) || type (1 B) || reply_to_length (4 B, big-endian) || reply_to (UTF-8) || deterministic JSON body.

  Byte strings inside the JSON body are encoded as hexadecimal under the $bytes marker.
- The wrapped_document_key field in the Share has the exact form: nonce (12 B) || ciphertext (32 B) || tag (16 B) This is because it protects one 32-byte document key.
- The RSA-PSS signature covers the Share signed bytes, including the document ID, sender and recipient IDs, both fingerprints, sequence number, timestamp, ephemeral P-256 public key, and the wrapped document key.

Type byte	Message	Purpose
16	REGISTER_REQUEST	Registration records from client to server
17	LOGIN_BEGIN_REQUEST	Start login for a user ID
18	LOGIN_CHALLENGE	One-time challenge, salt, and Argon2 parameters
19	LOGIN_FINISH_REQUEST	Client HMAC response to the login transcript
20	LOGIN_RESULT	Successful login result and account records
21	SESSION_REQUEST	Authenticated operation after login
22	SESSION_RESULT	Server result for registration/session operations
23	ERROR	Generic external failure response

2.4 Life of a Document: Upload to Verified Retrieval

1. **Registration:** the client derives password values, generates RSA-3072 and P-256 identity keys, wraps the private-key bundle, and sends only the serialized account records to the server.

2. **Login:** the server sends a one-time challenge with the account salt and Argon2id parameters. The client derives the login material and answers with HMAC-SHA-256 over the login transcript. Only after a valid response does the client recover the wrapped private-key bundle.

3. **Upload:** the client generates a fresh 32-byte document key, 12-byte nonce, and 16-byte document ID. The file is encrypted with ChaCha20-Poly1305. Filename, MIME type, size, timestamp, owner, and document ID are authenticated as associated data.

4. **Share:** the sender obtains the recipient's public key, applies TOFU/change detection, generates an ephemeral P-256 key pair, derives an ECDH secret and HKDF wrapping key, encrypts the document key, and signs the Share object's signed bytes with RSA-PSS.

5. **Receive:** the recipient checks the pinned sender key/fingerprint, verifies the RSA-PSS signature, checks freshness, derives the ECDH/HKDF wrapping key, and unwraps the document key.

6. **Download:** the client requests the encrypted Document object, checks the requested document ID, verifies the AEAD tag, and only then releases plaintext.

3. Justification of Section 7 Decisions

3.1 Protecting Passwords

Argon2id

Choice: Argon2id through argon2-cffi. The team considered PBKDF2, bcrypt, scrypt, Argon2id, and custom construction. The design notes selected Argon2id because it is memory-hard and allows us to tune memory, time cost, and parallelism. PBKDF2 is not memory-hard, bcrypt uses a small, fixed memory footprint, and scrypt was considered a valid memory-hard alternative.

Each account uses fresh 16-byte salt. The selected parameters are memory_cost = 131072 KiB (128 MiB), time_cost = 3, parallelism = 1, hash_len = 32, and version = 19. In our benchmark, Argon2id plus the two HKDF derivations had a median time of 0.2301 s on the test machine, which was the closest candidate to the 0.25 s target.

For an illustrative offline-guessing calculation, a uniformly random 8-character alphanumeric password has a search space of $62^8 \approx 2.18\times10^{14}$ values, so the average case is about 1.09×10^{14} guesses. At the measured sequential rate of $1/0.2301 \approx 4.35$ guesses/s, this is about 7.9×10^5 years on one process. This is only a baseline based on our machine and assumptions. A weak dictionary password has a much smaller effective search space.

HKDF then separates the password-derived material into an authentication value and a wrap key. A password change therefore needs a new credential record and a new wrapped-private-key record, but it keeps the same RSA and P-256 identity keys.

3.2 Protecting Document Contents

Choice: ChaCha20-Poly1305, implemented by the team. The design notes compared it with AES-GCM, AES-CCM, and manually composing encryption and authentication. We chose a standardized AEAD construction so that confidentiality and integrity are handled together.

In our implementation, counter 0 generates the one-time Poly1305 key, while encryption starts at counter 1. The authentication tag covers both the associated data and ciphertext, and the plaintext is released only after successful verification.

Each document uses a fresh 256-bit document key and a fresh 96-bit nonce generated from the operating system random source. The document key is used for one document encryption. The metadata remains readable by the server, but it is also included as associated data. Therefore, changing the metadata causes authentication to fail. Random nonces provide probabilistic uniqueness rather than an absolute guarantee.

3.3 Public-Key Cryptography

Choice: ECDH over P-256 for key agreement and RSA-PSS with a 3072-bit RSA key for signatures. The design target is approximately 128-bit classical security.

P-256 is used for key agreements and sharing. In our format, an uncompressed P-256 public key is 65 bytes. At approximately the same 128-bit classical security level, a 3072-bit finite-field value occupies 384 bytes, so the P-256 public value is about 5.9 times smaller. The supplied Part 3 benchmark results report averages over 10 runs of 0.07873 s for P-256 key generation and 0.08207 s for ECDH. These are measurements of our Python implementation, not universal performance claims. A measured finite-field Diffie-Hellman timing baseline was not available in our final benchmark results, so we do not claim a measured speed advantage.

RSA-PSS uses SHA-256, MGF1-SHA-256, and a fresh 32-byte random salt. The team selected RSA-PSS over ECDSA because the design notes preferred its failure mode: accidental nonce reuse in ECDSA can reveal the private key, while RSA-PSS does not depend on a secret per-message signing nonce in the same way. The tradeoff is larger RSA keys and 384-byte signatures.

For sharing, the sender generates an ephemeral P-256 key pair and performs ECDH with the recipient's long-term P-256 public key. HKDF-SHA-256 with the label securevault:v1:share-wrap converts the shared secret into the 32-byte key used to protect the document key.

3.4 Trusting a Public Key

Choice: Trust-on-first-use (TOFU) with fingerprint pinning and optional out-of-band comparison.

On first contact, the client records the other user's public-key fingerprint. Later key changes are rejected instead of being silently trusted. The fingerprint is calculated using SHA-256 over the canonical public-key record, which binds the user identity to both the P-256 and RSA public keys.

TOFU was chosen because it allows first contact without operating a certificate authority. Its limitation is that the first key is not authenticated by TOFU itself. An out-of-band fingerprint comparison is needed if users want stronger first-contact assurance.

3.5 Signatures Versus MACs

The project uses MACs when both sides already share a secret, and signatures when third-party verification is needed. HMAC-SHA-256 is used for login/session authentication and inside HKDF. Poly1305 authenticates the AEAD ciphertext. RSA-PSS is used on Share objects because the recipient can verify the signature using the sender's public key.

A MAC cannot provide non-repudiation because the verifier also holds the MAC key and could have generated the same tag. A digital signature can be shown to a third party, but only if the signing public key is reliably associated with the claimed user.

In the current implementation, the Share signature authenticates the share/key grant. It does not include a hash of the complete Document object, so full proof of the exact document contents is listed as a limitation.

3.6 Architecture and Formats

Choice: Client/server communication through files in a shared directory that simulates the network. This was an allowed architecture in the project specification. We kept the client/server model because it matches centralized storage and asynchronous sharing while allowing us to focus on the cryptographic behavior instead of socket code.

Objects use explicit version/type bytes, deterministic field ordering, 4-byte length prefixes for variable fields, and fixed widths for cryptographic values. Verification failures are reported through a limited set of messages, such as Authentication failed, Key mismatch, Verification failed, Stale object, or Malformed object.

Implementation / Library Declaration

Implemented by the team	Library / standard support
SHA-256; HMAC-SHA-256; HKDF; ChaCha20; Poly1305; ChaCha20-Poly1305 AEAD; RSA-3072 key generation; Miller-Rabin; MGF1; RSA-PSS; P-256 arithmetic/keygen/validation; ECDH; object formats; trust/freshness/share logic.	Argon2id via argon2-cffi. OS randomness, Python big integers, SQLite, JSON, file handling, and pytest are non-cryptographic support. Reference crypto libraries are used inside tests for comparison, not as the submitted primitive implementation.

4. Security Argument

The table below maps each required security property to the mechanism present in the final design.

Property	Mechanism in SecureVault	Important boundary

Confidentiality	ChaCha20-Poly1305 encrypts each document under a fresh 32-byte document key. Share ECDH/HKDF protects that document key for the recipient.	The trusted client may see plaintext; the server and transport do not receive the document key.
Integrity	Poly1305 tag verification happens before plaintext is accepted. The wrapped private bundle and wrapped document key are also AEADprotected.	Authentication failure must stop processing; unauthenticated plaintext is not used.
Data-origin authentication	RSA-PSS verifies the sender of a Share object after the sender's public key has been checked against the trust store.	This authenticates the share/key grant, subject to correct public-key binding.
Non-repudiation	RSA-PSS can be verified by a third party using the sender's public key.	Current signature does not commit to a hash of the full Document object, so full content-level non-repudiation is incomplete.
Credential protection	Fresh per-user salt + measured Argon2id work factor; passwordderived wrap key is not stored serverside.	Weak passwords remain guessable; the stored verifier is sensitive and can be used in the current login protocol.
Freshness	One-time login challenges, stored session request IDs, and persender/per-document share sequence numbers plus timestamps.	State must be retained; the first share has no previous value to compare against.
Metadata binding	Document metadata is AEAD associated data. Share identifiers, fingerprints, freshness values, ephemeral public key, and wrapped key are inside signed/AAD bytes.	Only fields actually covered by AAD/signature are protected against undetected modification.

Correctness Evidence

Implemented primitives are tested against published vectors or independent reference implementations. The project tests include RFC 4231 HMAC vectors, RFC 5869 HKDF vectors, RFC 8439 ChaCha20/Poly1305/AEAD vectors, RSA-PSS reference-library checks, and P-256 known-point/ECDH checks. Integration tests use the real client, server, shared-directory transport, and server-restart path. The final complete pytest suite passed 237 tests. In addition, the full-system test completed the real Layla-to-Omar flow and checked replay rejection, document and Share

tampering, TOFU public-key substitution, server-side plaintext exposure, and logout/session cleanup.

5. Limitations

Current limitation	What would be needed to improve it
TOFU does not authenticate the first public key.	Users compare fingerprints through an authenticated out-of-band channel, or the system adds a stronger certification mechanism.
The stored verifier is sufficient to participate in the current HMAC login check if the server database is stolen.	Use an asymmetric login proof or another protocol that does not make the stored verifier an authentication secret.
The Share signature authenticates the share/key grant but not a hash of the complete Document object.	Add SHA-256(Document) or an equivalent document commitment to the signed bytes.
The command-line client stores active session material, wrap key, and recovered identity keys in local session state.	Keep sensitive values in process memory or protected OS key storage in a production client.
Random nonces provide probabilistic, not mathematical, uniqueness.	Use a persistent nonce/counter strategy if an application requires a hard uniqueness guarantee across crashes/restarts.
The shared-directory transport is a project simulation rather than a deployed authenticated network channel.	Replace it with a real authenticated transport while keeping the same message/object verification rules.
The reply address in the application message envelope is not authenticated. A network attacker can redirect a response to another inbox.	Include the envelope routing fields, including the reply address, in the authenticated request data so that modifying them causes verification to fail.
Timing checks were measured in one local environment.	Repeat measurements on more machines and larger samples before making broader timing claims.
No measured finite-field DH comparison is present in the supplied benchmark results.	Run and record a comparable finite-field benchmark before claiming a timing advantage; until then, report only the measured P-256 values.

6. AI Usage Statement and Contributions

6.1 AI Usage Statement

ChatGPT (OpenAI) was used as a support tool during the project for debugging, understanding error messages, explaining concepts, checking syntax, organizing and formatting the documentation, arranging the report content, and preparing Markdown (md) files. It was also used to help review the wording of the report and presentation materials.

The cryptographic design, implementation decisions, protocol logic, and Section 7 decisions were made by the team. AI was not used to choose cryptographic algorithms or make the final design decisions. The team reviewed and tested the code and made the final changes themselves.

The benchmark results and test results included in this report were obtained by running our actual project code. We used AI only as a supporting tool during development and documentation, while the implementation work, testing, decisions, and final review were done by the team.

6.2 Contribution Statements

Member	Main contribution
Aseel Hamed — Part 1	RSA-3072 key generation and RSA-PSS; HMAC/HKDF; password derivation and Argon2id measurement; private-key wrapping/recovery; registration, login, sessions, password change, and related tests.
Ruaa Othman — Part 2	SHA-256; ChaCha20, Poly1305, and AEAD; common object formats; shared-directory transport; server/client storage and application integration; related tests.
Yaqeen Salameh — Part 3	P-256 arithmetic/key generation/validation; ECDH; TOFU trust; sharing; freshness/replay logic; related tests and project document assembly.

References

[1] ENCS4320 Applied Cryptography, Final Course Project Specification, Term 1253, Birzeit University.

[2] ENCS4320 Applied Cryptography, Course Lecture Slides, Birzeit University.

[3] National Institute of Standards and Technology (NIST), *Secure Hash Standard (SHS)*, FIPS PUB 180-4, August 2015. Available: https://doi.org/10.6028/NIST.FIPS.180-4

[4] H. Krawczyk and P. Eronen, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, 1997. Available: https://www.rfc-editor.org/rfc/rfc2104

[5] D. H. NIST, *Identifiers and Test Vectors for HMAC-SHA-224, HMAC-SHA-256, HMACSHA-384, and HMAC-SHA-512*, RFC 4231, IETF, 2005. Available: https://www.rfceditor.org/rfc/rfc4231

[6] H. Krawczyk and P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, IETF, 2010. Available: https://www.rfc-editor.org/rfc/rfc5869

[7] Y. Nir and A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, IETF, 2018. Available: https://www.rfc-editor.org/rfc/rfc8439

[8] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017, IETF, 2016. Available: https://www.rfceditor.org/rfc/rfc8017

[9] National Institute of Standards and Technology (NIST), *Recommendations for Discrete Logarithm-Based Cryptography: Elliptic Curve Domain Parameters*, NIST SP 800-186, February 2023. Available: https://doi.org/10.6028/NIST.SP.800-186

[10] A. Biryukov, D. Dinu, and D. Khovratovich, *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*, RFC 9106, IETF, 2021. Available: https://www.rfceditor.org/rfc/rfc9106

[11] argon2-cffi 25.1.0, Python Argon2id library, used for the Argon2id implementation.

[12] Python 3.13 Documentation, Python Software Foundation.

[13] SQLite Documentation, SQLite, used through Python's standard library.

[14] pytest Documentation, pytest, used for project testing.

[15] cryptography Python library, used only in test files to cross-check the team's cryptographic primitives; it is not part of the submitted implementation.